

# Optimising Evaluation of Arithmetic Statements

## Problem Statement

You are supposed to write 2 programs, one to parse the arithmetic statements and output reduced arithmetic statements

Another Program to parse reduced arithmetic statements and output a set of optimised reduced arithmetic statements (reduced form) with meaning identical to the original arithmetic expression that is, it has the identical output when interpreted.

You will be judged on the size of optimised reduced form and execution time of your program

1. Remove unnecessary definitions
2. Constant propagation

## Input: Arithmetic Statements

## Output I : Reduced Arithmetic Statements in form of Binary Operations and Print Statements

## Output II : Optimised Output II

### Input Format:

Multiple Arithmetic Statements, each in a new line.

### Structure of Arithmetic Statements:

VARIABLE = ARTH\_EXPR print VALUE

ARTH\_EXPR VALUE (BinOp VALUE) *BinOp is +, -, , /*

VALUE: INTEGER or VARIABLE

VARIABLE string of digits and characters, not beginning with a digit

INTEGER sequence of digits

### Reduced Arithmetic Statements

VARIABLE = Value BinOp Value VARIABLE = Value print VALUE

### Examples

Example 1

Input:

x = 2 \* 3 + 5 - 1

Output I:

```
t1 = 2 * 3
t2 = t1 + 5
t3 = t2 - 1
x = t3
```

Output II:

Example 2 Input:

```
x = 2 + 3 * y - 1
print x
```

Output I:

```
t1 = 2*3
t2 = t1 + y
t3 = t2 - 1
x = t3
print x
```

Output II:

```
p1 = 6 + y
x = p1 - 1
print x
```